

```
# Installation (run in terminal / command prompt)
```

```
!pip install numpy
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
```

```
# Standard import convention (always use 'np' alias)
```

```
import numpy as np
```

```
# Check version
```

```
print(np.__version__)
```

```
2.0.2
```

```
# 1D Array it will be referred as vector
```

```
array1d=np.array([1,2,3])
```

```
print(array1d)
```

```
[1 2 3]
```

```
# 2D Array it will be referred as Matrix
```

```
array2d=np.array([[1,2,3],[4,5,6],[7,8,9]])
```

```
print(array2d)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

```
# 3D Array it will be referred as tensor
```

```
array3d=np.array([[[1,2,3],[4,5,6],[7,8,9]],[[10,11,12],[13,14,15],[16,17,18]]])
```

```
print(array3d)
```

```
[[[ 1  2  3]
   [ 4  5  6]
   [ 7  8  9]]
```

```
 [[10 11 12]
  [13 14 15]
  [16 17 18]]]
```

1D VS 2D VS 3D

```
# This is formatted as code
```

Built-in Array Creation Functions

```
# ZEROS AND ONES
```

```
np.zeros((2,5))
```

```
"""
```

```
np.zeros() function will create a matrix with values of 0 based
on given parameters
```

```
"""
```

[Show hidden output](#)

```
np.ones((2,5))
```

```
"""
```

```
np.ones() function will create a matrix with values of 1 based
on given parameters
```

```
"""
```

```
'\nnp.ones() function will create a matrix with values of 1 based \non given parameters\n'
```

```
np.full((2,5),5)
```

```
"""
```

```
np.full() function will create a matrix with values of 0 based
on given parameters
```

```
"""
```

[Show hidden output](#)

```
#IDENTITY MATRIX
np.eye(4)
"""
np.eye() function will create a identity matrix with values of 0 and 1
"""

'\nnp.eye() function will create a identity matrix with values of 0 and 1\n'
```

```
#RANGES
np.arange(0,15,4)
"""
np.arange(start,stop,step)
"""

'\nnp.arange(start,stop,step)\n'
```

```
np.linspace(2,10,12)

array([ 2.          ,  2.72727273,  3.45454545,  4.18181818,  4.90909091,
        5.63636364,  6.36363636,  7.09090909,  7.81818182,  8.54545455,
        9.27272727, 10.          ])
```

Random Arrays

```
# Uniform Random[0,1]
np.random.rand(9,3)

array([[0.52447175, 0.07206643, 0.6313266 ],
       [0.01947662, 0.99189993, 0.23052007],
       [0.39674086, 0.33783592, 0.2744089 ],
       [0.62449467, 0.78402448, 0.74065336],
       [0.54927633, 0.41510097, 0.25948586],
       [0.41172209, 0.75439829, 0.19982319],
       [0.36933579, 0.64074985, 0.60506222],
       [0.70375396, 0.49089473, 0.08861889],
       [0.49643608, 0.83920267, 0.10590679]])
```

```
# Standard Normal Distribution
np.random.randn(3,3)

array([[ 1.01963331, -0.13655436,  0.15688647],
       [-1.0752201 ,  0.4228149 , -0.54407495],
       [-1.5625948 ,  1.03956801, -2.34778154]])
```

```
# Random Integers
np.random.randint(1,5,size=(4,4))

array([[2, 1, 2, 3],
       [2, 4, 1, 3],
       [2, 3, 2, 3],
       [3, 1, 3, 2]])
```

```
# Uninintialized ( fast, but values are garbage)
np.empty((3,5))

array([[6.71482317e-316, 0.00000000e+000, 0.00000000e+000,
        4.79243676e-322, 7.85215132e-316],
       [6.76254005e-310, 1.50884601e-142, 8.42241414e+252,
        1.67388222e+296, 2.42092166e-322],
       [6.76254005e-310, 6.76254005e-310, 6.76254005e-310,
        6.76254005e-310, 6.32404027e-322]])
```

Array Attributes

```
arr= np.array([["a","b","c","d","e"],[1,2,3,4,5]])
print(arr.shape)    # (2, 3) - rows x columns
print(arr.ndim)     # 2      - number of dimensions
print(arr.size)      # 6      - total number of elements
print(arr.dtype)     # int64  - data type
print(arr.itemsize)  # 8      - bytes per element
print(arr.nbytes)    # 48     - total bytes used
```

```
(2, 5)
2
10
<U21
```

```
84
840
```

Indexing and Slicing

```
# 1D INDEXING
a = np.array([10, 20, 30, 40, 50])

# Positive indexing (left to right, starts at 0)
print(a[0])    # 10
print(a[2])    # 30
print(a[-1])   # 50 (last element)

# Slicing: [start : stop : step]
print(a[1:4])  # [20 30 40]
print(a[:3])   # [10 20 30]
print(a[::2])  # [10 30 50]
print(a[::-1]) # [50 40 30 20 10] (reversed)
```

```
10
30
50
[20 30 40]
[10 20 30]
[10 30 50]
[50 40 30 20 10]
```

```
#2D INDEXING
m = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

# Single element: [row, col]
print(m[0, 0])  # 1
print(m[1, 2])  # 6

# Row slicing
print(m[0, :])  # [1 2 3] - first row
print(m[:, 1])  # [2 5 8] - second column

# Sub-matrix
print(m[0:2, 1:3]) # [[2 3] [5 6]]
```

```
1
6
[1 2 3]
[2 5 8]
[[2 3]
 [5 6]]
```

```
# Boolean / Fancy Indexing
a = np.array([10, 25, 8, 42, 17])

# Boolean mask
mask = a > 15
print(mask)    # [False  True False  True  True]
print(a[mask]) # [25 42 17]

# Shorthand
print(a[a > 15]) # [25 42 17]

# Fancy indexing (select by index list)
print(a[[0, 3]]) # [10 42]
```

```
[False  True False  True  True]
[25 42 17]
[25 42 17]
[10 42]
```

ARRAY OPERATIONS

```
# Element-wise Arithmetic
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b)  # [5 7 9]
print(a - b)  # [-3 -3 -3]
print(a * b)  # [ 4 10 18]
```

```
print(a / b) # [0.25 0.4 0.5]
print(a ** 2) # [1 4 9] - squaring
print(a + 10) # [11 12 13] - scalar broadcast
```

```
[5 7 9]
[-3 -3 -3]
[ 4 10 18]
[0.25 0.4 0.5 ]
[1 4 9]
[11 12 13]
```